SAVEETHA SCHOOL OF ENGINEERING
C PROGRAMMING ASSIGNMENT – Logic Analysis and Design Challenges

Field	Entry
Student Name	BAVADARANI S
Register Number	1912260562
Course Code & Name	CSA0209 C PROGRAMMING
Slot	B
Assignment / Task Title	Logic Analysis and Design Challenges (Q1–Q7, 50 marks)
GitHub Repository URL	https://github.com/bavadarini28 /CSA0209-1912260562-BAVADARANI.S.git
Submission Date	12 NOVEMBER 2026

Q1 – Precedence, Associativity and the Discount Bug

Predictions before running (by the rules)

Precedence (high → low): * / (same level, left-to-right), then + − (left-to-right), then > , then ?: (right-to-left), then = (right-to-left). The rule I apply to total = price*qty - price*qty*15/100*senior + price*qty*5/100 is:

```
total = ((price*qty) - ((((price*qty)*15)/100)*senior)) + (((price*qty)*5)/100)
```

Order: (1) price*qty; (2) ×15; (3) integer-divide by 100 (the fraction is truncated); (4) ×senior; the subtraction is then done; (5) the right-hand term price*qty*5/100 is built the same way; (6) the final + is evaluated left to right; (7) > compares total with 1000; (8) ?: picks total−50 or total; (9) = stores the result.

No.	price, qty, senior	Predicted total	Predicted net	Observed total / net	Pharmacist intended (net)
1	250, 4, 1	900	900	900 / 900	Rs 892.50
2	120, 5, 0	630	630	630 / 630	Rs 630.00
3	999, 1, 1	899 (999 − 149 + 49)	899	899 / 899	Rs 891.61
4	500, 3, 1	1350	1300	(by hand) 1350 / 1300	Rs 1288.75

Bill 3 by hand: 999*15 = 14985; /100 = 149 (true 149.85); ×1 = 149. GST term 999*5 = 4995; /100 = 49 (true 49.95). Total = 999 − 149 + 49 = 899. Bill 4: discount 1500*15/100 = 225, GST term 1500*5/100 = 75, total = 1350 > 1000, so net = 1300.

Expressions that differ from what the pharmacist intended

- **GST is charged on the undiscounted amount.** The code adds 5% of price*qty; the intended rule (assumption: GST applies to the discounted value) is 5% of price*qty − discount. Bill 1: 900 vs 892.50.
- **Integer division truncates paise.** 15/100 style divisions drop the fraction (149.85 → 149, 49.95 → 49). Writing price*qty*(15/100) would be even worse: 15/100 = 0, so the discount would vanish.
- **The Rs 50 rebate uses the wrong total,** so bills near Rs 1000 can fall on the wrong side of the threshold (e.g. buggy total 1000, correct total 1050).
- **Money is held in int rupees;** no paise can be shown.

Inquiry – count = count++ + 1

The statement both modifies count (the ++) and reads count (the + 1 operand) with no sequence point between them, so the two accesses are unsequenced. The C standard says the behaviour is undefined: the program may give 6, 7 or anything. On my gcc 13 it printed 6, but another compiler or optimisation level may differ. With -Wall gcc reports:

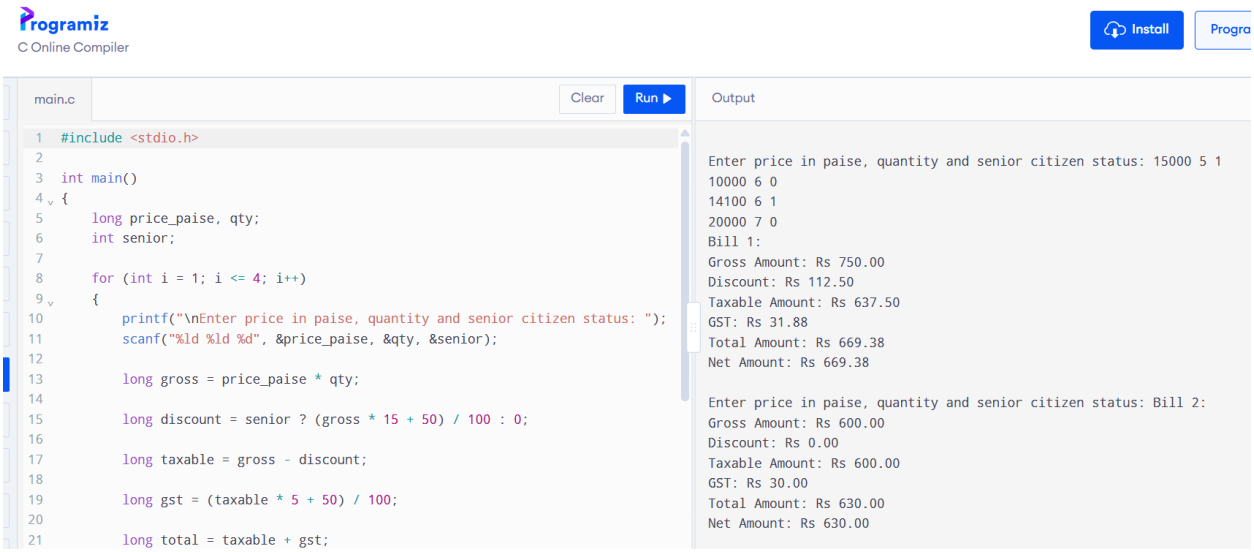
```
q1.c:13:21: warning: operation on 'count' may be undefined [-Wsequence-point]
```

Fix: write count = count + 1; or count++; (never both).

Design decision – corrected calculation (paise, brackets and intermediate variables)

```
long gross   = (long)price_paise * qty;
long discount = senior ? (gross * 15 + 50) / 100 : 0;   /* round half-up */
long taxable  = gross - discount;
long gst      = (taxable * 5 + 50) / 100;
long total    = taxable + gst;
long net      = (total > 100000L) ? (total - 5000L) : total;  /* Rs 1000, Rs 50 */
```

Output for the four bills: Rs 892.50, Rs 630.00, Rs 891.61 and net Rs 1288.75 – matching the hand-calculated intended values. I use both: intermediate variables so that each business rule (gross, discount, GST, rebate) is a visible, testable line, and brackets only where precedence is not obvious to a reader (e.g. (gross * 15 + 50) / 100). Paise in long removes the truncation error.



Alternative Concept and Complexity Analysis

Several statements with temporaries gross, discount, tax. Time: the same arithmetic operations (about 3 multiplications, 3 divisions, 3 additions) – compilers typically produce identical machine code, so O(1) either way. Space: 3–5 extra variables, i.e. 24–40 bytes of stack, usually in registers: O(1). Correctness improves because the order of evaluation is explicit and each step can be printed and checked; readability improves because the code mirrors the business rules, and a bug such as the GST base is visible at once.

Q2 – When Does Short-Circuit Evaluation Matter?

Experiment design

1,000 passengers generated with srand(42): about 30% senior, 50% eligible class, 5% hold a card. slow(i) increments a global call counter. Four formulations of the same rule were run on the same data.

Formulation	Calls to slow()	Granted	Why
(S && E) || slow() [cheap first]	836	211	slow() runs only when (S && E) is false: 1000

Formulation	Calls to slow()	Granted	Why
			− 164
slow() || (S && E) [slow first]	1000	211	slow() always runs first
(S && E) | slow() [non-short-circuit]	1000	211	| evaluates both sides always
Nested if / else if (cheap first)	836	211	else-branch only reached when the first test fails

Operand not evaluated: in a && b, b is skipped when a is false; in a || b, b is skipped when a is true. Test: x=0; (0 && ++x) leaves x = 0, (1 || ++x) leaves x = 0; but with & and | the ++x always runs and x becomes 1.

& versus &&: & is a bitwise operator: both operands are always evaluated, with no guaranteed order, and on arbitrary integers (e.g. 2 & 1 = 0 although both are "true"). && yields 0 or 1 and short-circuits with a sequence point after the first operand.

Rule: for &&, put first the cheapest condition that is most likely false; for ||, the cheapest condition most likely true. Order is irrelevant when both operands are cheap and free of side effects (the saving is a few cycles), when the first operand is always decisive/never decisive, or when every operand must be evaluated anyway. Order is also functionally important when the first test guards the second (p != NULL && p->x).

Changed condition – audit log

Measured: with the cheap-first order only 836 of the 1000 passengers were logged (164 were never verified or recorded); with slow-first all 1000 were logged. The function now has a side effect that must happen for every passenger, so short-circuit skipping is a correctness bug, not an optimisation. Recommendation: call the verifier unconditionally and store the result (int card = slow_log(i);), then evaluate (S && E) || card. The number of calls is then n regardless of order, so the ordering optimisation no longer applies. If only some passengers must be audited, keep the cheap-first order and document that unaudited cases are intentional.

Alternative Concept and Complexity Analysis

Nested if statements behave like && / ||: the slow function is called only when the cheap test fails – expected calls n(1 − p) where p = P(S && E) (836 here), worst case n, so O(n) with a smaller constant. With & and | the slow function is always called: exactly n calls (1000), plus no protection for guarded expressions. Space is O(1) for all three.

Q3 – Device-Status Register with Bitwise Operators

Predicted register values (before running)

No.	Statement in main()	Predicted value (binary)	Decimal
1	reg = 0	00000000	0
2	switch_on(reg, LIGHT)	00000001	1
3	switch_on(reg, AC)	00000101	5
4	toggle(reg, FAN)	00000111	7
5	switch_off(reg, LIGHT)	00000110	6

The program prints the register after statements 3 and 5, so the output is 00000101 and 00000110, followed by AC is ON, PUMP is OFF.

Complete the Code – TODO 1 to TODO 4

```
unsigned char switch_on(unsigned char reg, int dev)  { return reg | (1u << dev); }
```

```
unsigned char switch_off(unsigned char reg, int dev) { return reg & ~(1u << dev); }
unsigned char toggle(unsigned char reg, int dev)     { return reg ^ (1u << dev); }
int is_on(unsigned char reg, int dev)                { return (reg >> dev) & 1u; }
/* counting: Kernighan's trick, each pass clears the lowest set bit */
int count_on(unsigned char reg){ int c = 0; while (reg) { reg &= reg - 1; c++; } return c; }
```

Output:

```
00000101
00000110
AC is ON, PUMP is OFF
devices on = 2
```

Programiz
C Online Compiler

main.c | Clear | Run ▶ | Output

```c
#include <stdio.h>

unsigned char switch_on(unsigned char reg, int dev)
{
    return reg | (1u << dev);
}

unsigned char switch_off(unsigned char reg, int dev)
{
    return reg & ~(1u << dev);
}

unsigned char toggle(unsigned char reg, int dev)
{
    return reg ^ (1u << dev);
}

int is_on(unsigned char reg, int dev)
{
    return (reg >> dev) & 1u;
}

```

```
00000101
00000110
AC is ON, PUMP is OFF
devices on = 2
```

Inquiry

- **reg & 1 << dev** – << has higher precedence than &, so it means reg & (1 << dev): it tests bit dev. For reg = 00000110, dev = 2 it gives 4 (non-zero = ON).
- **(reg & 1) << dev** – first isolates bit 0 (LIGHT), then moves that bit to position dev. For the same reg it gives 0 because LIGHT is off. It is a different question, so always add brackets.
- **signed char with bit 7 set:** 0x80 is −128; it is promoted to int with sign extension (0xFFFFFF80). Measured: sc >> 7 = −1 (arithmetic shift, implementation-defined for negatives) versus 1 for unsigned char; sc == 0x80 is false because −128 ≠ 128 (gcc warns -Wtype-limits). Use unsigned char for bit registers.

Alternative Concept and Complexity Analysis

```
char state[8];
state[dev] = 1;      /* on  */
state[dev] = 0;      /* off */
state[dev] ^= 1;     /* toggle */
return state[dev];   /* test */
int count=0; for (int i=0;i<8;i++) count += state[i];
```

Every single-device operation is O(1) in both designs. Counting the devices that are ON: the array needs a loop of 8 steps, O(d) for d devices; the bit register needs at most one pass per set bit (Kernighan), or a 256-entry lookup table in O(1). Space for 1,000,000 homes: 1 byte per home = 1 MB with the register versus 8 bytes per

home = 8 MB with the array (8× more). The array is simpler and allows storing more than ON/OFF per device (e.g. a dimmer level), but the register also lets one AND/OR change many devices at once.

Q4 – else–if Ladder or switch?

Test results

```
char grade(int marks)
{
   if (marks < 0 || marks > 100) return '?';
   switch (marks / 10) {
      case 10: case 9: return 'S';
      case 8: return 'A';
      case 7: return 'B';
      case 6: return 'C';
      case 5: return 'D';
      default: return 'F';
   }
}
```

Output: 100:S 95:S 83:A 50:D 49:F -5:?

Marks	else–if	switch (guarded)	switch without guard	Comment
100	S	S	S	case 10 needed (100/10)
90 / 89	S / A	S / A	S / A	boundary correct
80 / 79	A / B	A / B	A / B	boundary correct
50 / 49	D / F	D / F	D / F	boundary correct
0	F	F	F	lowest valid
−1	?	?	F (wrong)	−1/10 = 0 (truncation toward zero)
101	?	?	S (wrong)	101/10 = 10 → case 10

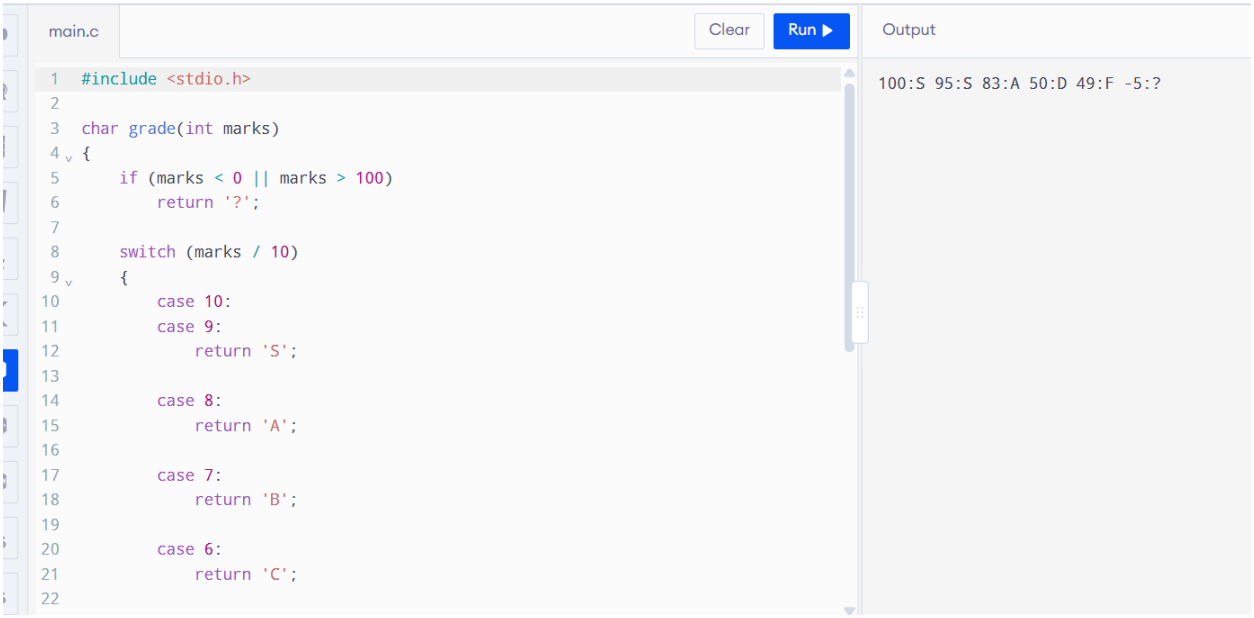
Missing break: removing the break after case 8 (so it falls into case 7) changed 83, 89 and 80 from A to B (gcc -Wimplicit-fallthrough warned). Removing a return gives the same fall-through effect: execution continues into the next case.

Ranges: a case label must be one constant integer, so 75–84 would need ten labels; marks / 10 can only express boundaries that are multiples of 10. (case 75 ... 84 is a non-standard GCC extension.)

Easier to change: the ladder – edit one number. The switch must be rewritten, and breaks as soon as a boundary is not a multiple of 10. A lookup table is also easy as long as the boundaries share a step.

Boundary-exposing test set: for every boundary b test b and b − 1: 100, 90, 89, 80, 79, 70, 69, 60, 59, 50, 49, 0; and the invalid neighbours −1, 101; plus extremes INT_MIN and INT_MAX.

Programiz
C Online Compiler



Design decision – scholarship rule

```
/* rows: grade S,A,B,C,D,F ; columns: income band 0 (low), 1 (middle), 2 (high) */
static const char *award[6][3] = {
  {"Full","Full","Half"}, {"Full","Half","Quarter"}, {"Half","Quarter","None"},
  {"Quarter","None","None"}, {"None","None","None"}, {"None","None","None"} };
const char *a = award[grade_index][income_band];
```

(The award values are illustrative.) I choose the lookup table: the rule is data, not code, so the board can audit or change it without touching control flow; it is O(1); and a 6 × 3 table is far easier to check than 18 nested if branches. A switch inside an if is acceptable for 2–3 rules but grows quickly with two dimensions; nested if is best only when conditions are not table-shaped (ranges, exceptions).

Alternative Concept and Complexity Analysis

```
static const char grades[11] = {'F','F','F','F','F','D','C','B','A','S','S'};
char g = (m < 0 || m > 100) ? '?' : grades[m / 10];
```

Time: one division and one array read, O(1). The ladder needs up to k comparisons (k = 5 here), O(k); a switch is O(1) when the compiler builds a jump table, O(log k) or O(k) otherwise. Space: 11 bytes for the table (plus the 6 × 3 pointer table for scholarships) versus no data for the ladder. The table has the same weakness as the switch: it needs equal-width ranges.

Q5 – Count the Work Done by a Loop

Deriving the loop from the principle

If n = a × b is composite, one factor satisfies a ≤ √n. So a composite n always has a divisor i with i × i ≤ n; it is enough to try i = 2, 3, … while i*i <= n, which needs no sqrt(). The loop stops early with return 0 at the first i for which n % i == 0; if the loop ends, no divisor exists and n is prime. checks++ is placed immediately before each division.

Complete the Code – TODO 1

```
  for (unsigned long i = 2; i * i <= n; i++) {
     checks++;
     if (n % i == 0) return 0;
  }
```

```
    return 1;
```

Output:

```
    97 prime     divisions = 8
   100 not prime divisions = 1
  7919 prime     divisions = 87
1000003 prime     divisions = 999
```

Hand check: 97 → i = 2…9 (9×9 = 81 ≤ 97, 10×10 = 100 > 97) = 8; 1000003 → i = 2…1000 = 999.

Programiz
C Online Compiler

main.c — Clear — Run ▶ — Output

```c
#include <stdio.h>

int isPrime(unsigned long n, int *checks)
{
    *checks = 0;

    if (n < 2)
        return 0;

    for (unsigned long i = 2; i * i <= n; i++)
    {
        (*checks)++;

        if (n % i == 0)
            return 0;
    }

    return 1;
}

int main()
{
```

```
    97 prime     divisions = 8
   100 not prime divisions = 1
  7919 prime     divisions = 87
1000003 prime     divisions = 999
```

n	Result	Improved for	while	do–while	i < n version
97	prime	8	8	8	95
100	not prime	1	1	1	1
7919	prime	87	87	87	7,917
1,000,003	prime	999	999	999	1,000,001

```
/* while */      unsigned long i = 2; while (i <= n / i) { checks++; if (n % i == 0) return 0; i++; }
/* do-while */   if (n < 4) return n >= 2;   /* guard BEFORE the loop */
                 unsigned long i = 2; do { checks++; if (n % i == 0) return 0; i++; } while (i <= n / i);
```

Inquiry

- **do–while with n = 2 or 3:** the body runs once before the condition is tested. For n = 2, 2 % 2 == 0, so 2 is wrongly reported not prime (measured: 0). For n = 3 the answer 1 is right only by luck, after a needless division. Hence the guard n < 4 before the loop.
- **Overflow of i * i:** with a 32-bit unsigned type and n near 4,294,967,291, when i reaches 65,536, i*i = 2^32 wraps to 0 (measured), so i*i <= n stays true and the loop gives wrong answers or runs on. i <= n / i never overflows. On 64-bit unsigned long it only happens for n above about 1.8 × 10^19, but the safe form costs nothing.

Changing the condition to i < n gives 95, 1, 7,917 and 1,000,001 divisions (table above): linear in n instead of √n.

Alternative Concept and Complexity Analysis

```
char *comp = calloc(N+1, 1);
for (unsigned long i = 2; i*i <= N; i++)
   if (!comp[i]) for (unsigned long j = i*i; j <= N; j += i) comp[j] = 1;
/* query: prime iff n >= 2 && !comp[n] */
```

Sieve: O(N log log N) once (measured 2,122,051 marking operations for N = 1,000,003; 78,499 primes found), O(1) per query, O(N) space (1,000,004 bytes). Trial division: O(√n) per query (999 divisions for the worst ticket), O(1) space. For a few tickets trial division is cheaper (999 vs 2.1 million operations); for many queries the sieve wins – e.g. testing every number up to 1,000,003 takes 67,741,504 divisions by trial division, but only about 2.1 million marks with the sieve.

Q6 – Loops That Must Terminate: Loan Repayment Schedule

Loop design

- **Which loop:** while. The number of months is not known beforehand (rules out for), and a loan with zero balance should print nothing (rules out do–while, which always executes once).
- **Termination:** while (balance > 0 && month < MAX_MONTHS). Money is in integer paise, so balance reaches exactly 0. With floating point, balance != 0 is dangerous: for example, 1.0 minus 0.1 ten times leaves 1.39 × 10^-16 in double (measured), never exactly 0, so the loop would not stop or would pay a 0.00000000000000014 instalment.
- **Last month:** pay min(balance, repayment). Otherwise the balance would turn negative.
- **Repayment ≤ monthly interest:** the balance never falls (it stays the same or grows), so the loop never ends. The program tests this before the loop and prints an error; the safety limit is a second line of defence.

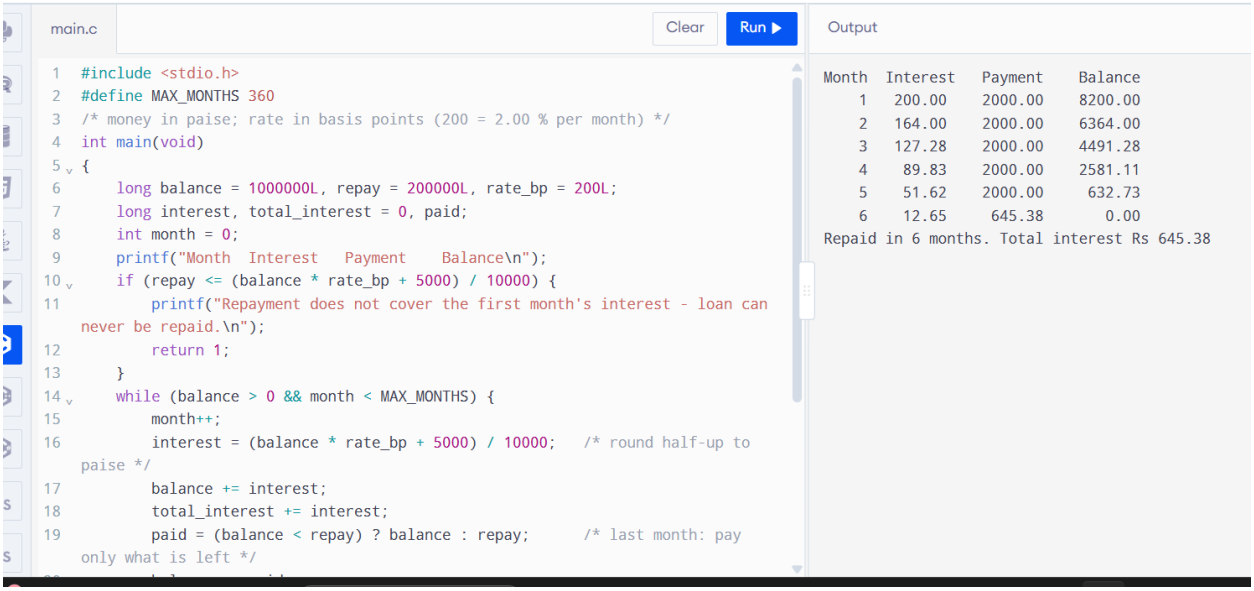
```
#include <stdio.h>
#define MAX_MONTHS 360
/* money in paise; rate in basis points (200 = 2.00 % per month) */
int main(void)
{
   long balance = 1000000L, repay = 200000L, rate_bp = 200L;
   long interest, total_interest = 0, paid;
   int month = 0;
   printf("Month  Interest   Payment    Balance\n");
   if (repay <= (balance * rate_bp + 5000) / 10000) {
      printf("Repayment does not cover the first month's interest - loan can never be repaid.\n");
      return 1;
   }
   while (balance > 0 && month < MAX_MONTHS) {
      month++;
      interest = (balance * rate_bp + 5000) / 10000;   /* round half-up to paise */
      balance += interest;
      total_interest += interest;
      paid = (balance < repay) ? balance : repay;      /* last month: pay only what is left */
      balance -= paid;
      printf("%5d %5ld.%02ld %7ld.%02ld %7ld.%02ld\n", month, interest/100, interest%100, paid/100, paid%100, balance/100,
balance%100);
   }
   if (balance > 0) printf("Safety limit of %d months reached; balance Rs %ld.%02ld still due.\n", MAX_MONTHS, balance/100,
balance%100);
   else printf("Repaid in %d months. Total interest Rs %ld.%02ld\n", month, total_interest/100, total_interest%100);
   return 0;
}
```

Output (Rs 10,000, 2% per month, Rs 2,000 repayment):

```
Month  Interest   Payment    Balance
   1   200.00    2000.00    8200.00
   2   164.00    2000.00    6364.00
   3   127.28    2000.00    4491.28
   4    89.83    2000.00    2581.11
```

```
  5   51.62    2000.00     632.73
  6   12.65     645.38       0.00
Repaid in 6 months. Total interest Rs 645.38
```

Programiz
C Online Compiler



Correctness check by hand

Month 1: 10000 × 0.02 = 200; 10200 − 2000 = 8200. Month 2: 164; 8364 − 2000 = 6364. Month 3: 127.28; 6491.28 − 2000 = 4491.28. Month 4: 89.8256; 4581.1056 − 2000 = 2581.1056. Month 5: 51.6221; 2632.7277 − 2000 = 632.7277. Month 6: 12.6546; balance 645.3823 < 2000, so the last payment is Rs 645.38. Total interest = 645.38 (exact 645.3823) – matches the program (the program rounds each month's interest to the paisa, hence the tiny differences in the 4th digit). Check: 5 × 2000 + 645.38 = 10,645.38 = 10,000 + 645.38.

Growth investigation

Prediction: months n = −ln(1 − rP/E) / ln(1+r) with rP = Rs 200 is the monthly interest on the first month; n grows without bound as E → 200. Measured (P = 10,000, r = 2%):

Repayment E (Rs)	Months (loop)	Months (formula)
2000	6	5.32
1000	12	11.27
500	26	25.80
300	56	55.48
250	82	81.27
220	122	121.09
205	188	187.53
201	268	267.81
200 or less	never ends	undefined

The count roughly doubles each time E − 200 halves; near the threshold a Rs 1 change in repayment changes the loan term by months.

Deployment decision

The limit is MAX_MONTHS = 360 (30 years: a longer micro-loan is not realistic). When it is reached the program prints: "Safety limit of 360 months reached; balance Rs … still due." and exits with an error status, not a "fully repaid" report. Defence: silent truncation would give a false schedule; the message tells the group leader to raise the repayment. Cases with repayment ≤ interest are rejected up front with a clear message, so the limit is only reached by very low (but valid) repayments.

Alternative Concept and Complexity Analysis

Closed form: months = ceil(−log(1 − r·P/E) / log(1 + r)) in O(1) time and O(1) space, versus O(n) iterations (6 here; up to 360). Precision: the formula uses floating-point logs, so the result is fractional and must be rounded up; it ignores the per-month rounding to paise, so the result can be off by a month at the boundary; it is undefined when r·P/E ≥ 1 (log of zero or a negative number) and loses accuracy when r·P/E is close to 1. It also cannot give the schedule, the final payment or the total interest without more work. The loop is exact in integer paise and produces the whole schedule, at a cost that is trivial for n ≤ 360.

Q7 – Diagnose, Redesign and Validate (chosen program: Q5 prime testing)

A. Diagnose the Bottleneck

Evidence from Q5: for a prime ticket such as 1,000,003 the loop makes 999 divisions, all inside the single loop in is_prime. The count grows with √n (97 → 8, 7919 → 87, 1,000,003 → 999). Verifying every number from 2 to 1,000,003 costs 67,741,504 divisions. Work is repeated unnecessarily: after n % 2 fails, every other even divisor (4, 6, 8 …) cannot divide n either – about half of the divisions are wasted; likewise multiples of 3 (another third). Composite numbers with a small factor (like 100) already exit after one division, so the cost is dominated by primes and by numbers with only large factors.

B. Design Objective

Objective: reduce the number of divisions for large primes (e.g. 1,000,003) by at least 50% and the total for all n ≤ 1,000,003 by at least 45%, with identical results (zero mismatches against the original over 2..1,000,003) and no extra memory.

C. Revised Design

Property exploited: every prime greater than 3 has the form 6k ± 1 (else it is divisible by 2 or 3). Version 1 skips even divisors; Version 2 tests 2 and 3 once, then only candidates 5, 7, 11, 13, 17, 19 … (i and i + 2 with i += 6). Extra memory: none. Trade-off: slightly more code, special cases for n < 4, and one extra test for 2 and 3 per call.

```
is_prime(n):
   if n < 2 return 0 ; if n < 4 return 1
   if n % 2 == 0 return 0 ; if n % 3 == 0 return 0
   i = 5
   while i * i <= n:
      if n % i == 0 or n % (i + 2) == 0: return 0
      i = i + 6
   return 1
int is_prime_6k(unsigned long n)
{
   checks = 0;
   if (n < 2) return 0;
   if (n < 4) return 1;                    /* 2 and 3 */
   checks++; if (n % 2 == 0) return 0;
   checks++; if (n % 3 == 0) return 0;
   for (unsigned long i = 5; i * i <= n; i += 6) {
      checks++; if (n % i == 0)       return 0;   /* 6k - 1 */
      checks++; if (n % (i + 2) == 0) return 0;   /* 6k + 1 */
   }
```

```
    return 1;
}
```

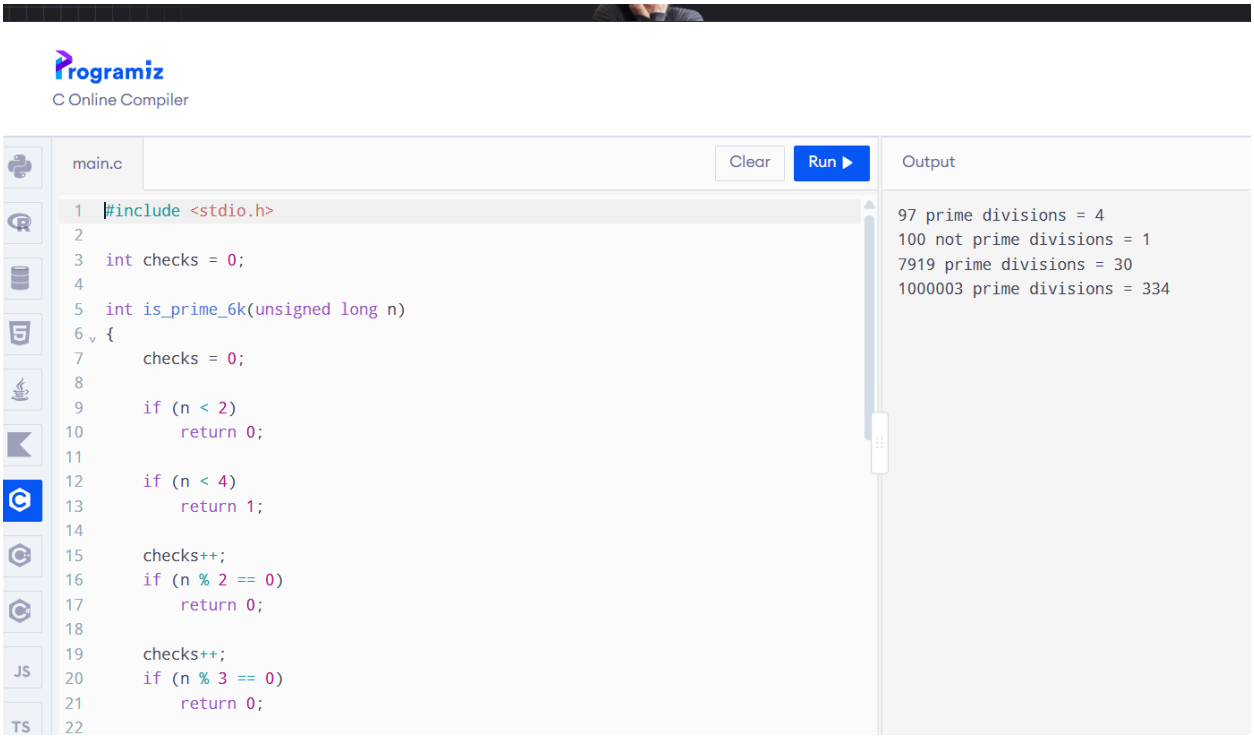
D. Prediction

For 1,000,003: original 999 divisions; odd-only ≈ 999 / 2 ≈ 500 (−50%); 6k ± 1 ≈ 999 / 3 ≈ 333–334 (−67%). Total over all n: roughly −50% and −66%. Measure: number of divisions (and time as a secondary measure). For small or even n (like 100) I expect no gain.

E. Implementation and Validation

n	Prime?	Original	Odd only	6k ± 1
97	yes	8	5	4
100	no	1	1	1
7,919	yes	87	44	30
1,000,003	yes	999	500	334
999,983	yes	998	500	334
1,000,001	no	100	51	35
All n = 2… 1,000,003	78,499 primes	67,741,504	34,601,109	23,739,568
Time for all n (-O0)	–	0.213 s	0.109 s	0.074 s

All three versions agreed on every n from 2 to 1,000,003 (0 mismatches) and found 78,499 primes, which also equals the number found by the sieve. Times are from one run on my machine without optimisation; yours will differ.



F. Defence

- Objective achieved: 999 → 500 (−50.0%) and → 334 (−66.6%) divisions for the large prime; for all n, −48.9% (odd only) and −65.0% (6k ± 1). The odd-only total misses 50% slightly because it spends one extra test on n % 2, and because composites that already exit at i = 2 gain nothing.
- Measured time fell from 0.213 s to 0.074 s (about 2.9× faster), which tracks the drop in divisions (67.7M → 23.7M, about 2.9× fewer): division dominates the run time, so counting divisions is a good proxy for speed.
- New costs: more complex code and special cases (n < 4); more places for an off-by-one error; no extra memory.
- Where the original may still be better: for a few small tickets (n = 100 gains nothing; 97 saves only 4 divisions) its simplicity and obvious correctness outweigh the saving. If many tickets are verified, the sieve (about 2.1 million marking operations and 1 MB for N = 1,000,003) beats all trial-division versions (23.7 million divisions for 6k ± 1), at the price of O(N) memory and a fixed upper limit.
- Conclusion: adopt the 6k ± 1 version for on-demand checks; use the sieve for batch verification of the whole range.

Feedback (CO2)

This assignment helped me apply operators, decision statements and loops to real problems (CO2). I learned to predict results from precedence rules, test boundary values, and judge a design by measured counts, and I reduced the prime-check divisions by about 65% with a 6k±1 redesign.